

Networks and Systems

BSc Computer Science - Durham University

Matthew Emerson

October 2026

Contents

1	Module Introduction	1
1.1	Assessment	1
1.2	Timetable	1
2	Cyber Security	1
2.1	Introduction, Threat Modelling, and Applied Cryptography	1
3	Networks	4
4	Databases	4
4.1	Enhanced Entity-Relationship Model	4
4.2	Semistructured Databases (XML)	4
4.3	XML Data Manipulation (XPath and XQuery)	4
4.4	Transactions	4
4.5	Concurrency Control	4
4.6	Distributed Transactions	4
4.7	Distributed Concurrency Control	4
5	Distributed Systems	4
5.1	Introduction to Distributed Systems	4
5.2	Middleware and RMI Technologies	4
5.3	Message-Oriented Middleware and Web Services	4
5.4	Replication Models	4
5.5	Concurrent Events and Consistency Control	4
5.6	Fault Tolerance	4
5.7	Load Distribution	4
I	Appendix	5
6	Bibliography	5
6.1	Module Professors	5
6.2	References	5
6.3	List of Figures	5
6.4	Reading List	5

1 Module Introduction

Networks and Systems has the module code COMP2211 and is led by Dr Wanqing Tu.

1.1 Assessment

This module is assessed through two pieces of summative coursework and two final exams, these can be seen in Table 1. Additional formatives are also shown in *italics*.

Title	Opening	Due	Feedback	Professor	Weight
Cyber Security	9th Oct	5th Nov	3rd Dec	Dr Carlton Shepherd	25%
Networks	23rd Oct	21st Jan	18th Feb	Dr Wanqing Tu	25%
Dist. Sys. & Databases	May Exam	-	-	Dr Vasantam	50%

Table 1: Assessments for Networks and Systems.

1.2 Timetable

This module is taught through one two-hour lecture and one two-hour computer class each week as seen below in Table 2.

Day	Time	Class Type	Location	Lecturer
Thursday	09:00 - 11:00	Lecture	CLC202	Dr Carlton Shepherd
Thursday	16:00 - 18:00	Computer Class	CC-0007	-

Table 2: Michaelmas Term Timetable for Networks and Systems.

2 Cyber Security

This section is taught by Dr Carlton Shepherd. The recommended reading for this module is as follows:

- Practical Cryptography for Developers (Nakov, Stefanov and Shideroff, 2018), specifically the section on “Encryption: Symmetric and Asymmetric”.

2.1 Introduction, Threat Modelling, and Applied Cryptography

2.1.1 Basic Definitions

Definition 2.1.1.1: A system’s security depends on the definition of the adversary: who they are, what they can do (or their capabilities) and their determination (time, money, resources). This is the adversary or threat model.

Definition 2.1.1.2: An asset is something that is worth protecting.

Definition 2.1.1.3: A threat is a person or program that may try and access/steal your assets.

Definition 2.1.1.4: An assumption is something you are relying on to be true/secure by default.

Definition 2.1.1.5: Your goal should be in terms of what needs to be secure, and how long for.

Definition 2.1.1.6: The controls are the things you can do to attempt to protect your assets.

The CIA triad are the three main security properties, these are:

- **Definition 2.1.1.7:** Confidentiality: The outcomes of different actions in the system are only visible by authorised subjects.
- **Definition 2.1.1.8:** Integrity: Actions modifying the system can only be performed by authorised subjects.

- **Definition 2.1.1.9:** Availability: Subjects can perform all actions they are authorised for.

In addition to these properties, it is also important to consider additional properties, such as:

- **Definition 2.1.1.10:** Authenticity: An entity is who they claim they claim to be.
- **Definition 2.1.1.11:** Non-Repudiation: An actor cannot easily deny that an action has occurred.
- **Definition 2.1.1.12:** Context-Binding: An action is valid only for a particular period of time, distance, location, and/or environment.

Definition 2.1.1.13: An exploit is a piece of code, a tool, or a technique that takes advantage of a flaw in software, hardware, or a network to cause unintended behaviour.

Definition 2.1.1.14: A vulnerability is a weakness or flaw in a system's design, implementation, operation, or human behaviour that an attacker can exploit to compromise security.

Definition 2.1.1.15: Risk is the probability or likelihood that a threat will exploit a vulnerability and cause harm, loss, or negative consequences to an asset.

2.1.2 STRIDE Modelling

Definition 2.1.2.1: One method of classifying security risks is STRIDE, which was first developed by Microsoft. It is separated as follows:

- **Definition 2.1.2.1:** Spoofing is the act of using someone else's credentials to perform an operation and is a failure of identification and authentication.
- **Definition 2.1.2.2:** Tampering is the malicious modification of data and is a failure of integrity and authenticity.
- **Definition 2.1.2.3:** Repudiation is the act of denying having performed an operation and is a failure of accountability and identification.
- **Definition 2.1.2.4:** Information Disclosure is the exposure of information to unauthorised users and is a failure of confidentiality.
- **Definition 2.1.2.5:** Denial of Service is the act of denying service to legitimate users and is a failure of availability.
- **Definition 2.1.2.6:** Elevation of Privilege is an instance of an unprivileged user gaining privileged access and is a failure of confinement and authentication.

This is a helpful model for structuring the analysis of potential system threats but it is not exhaustive or exclusive.

2.1.3 Applied Cryptography

Definition 2.1.3.1: To encode some given data, it is transformed from one format/domain to another. This is fully reversible, and does not provide any secrecy. The inverse procedure is called decoding.

- **Definition 2.1.3.2:** A subtype of encoding is steganography, where the existence of the secret message is concealed.

Definition 2.1.3.3: To hash some given data, it is transformed into a (usually) smaller, fixed-size. This is most commonly considered a one-way function. An example of this are SHA algorithms.

- A hashing algorithm works by taking a message m and applying a hashing algorithm $h = H(m)$. h can then be sent securely and the recipient can then take $h' = H(m')$ where $m = m'$, and if $h' = h$, then the message received is correct.

Definition 2.1.3.4: Non-secret data (plaintext) can be encrypted into ciphertext using a secret key. The output is $Enc(m, K) = c$. The inverse procedure, decryption, again uses a secret key to transform the ciphertext back to plaintext $Dec(c, K) = m$.

2.1.4 Cipher Examples

Definition 2.1.4.1: A substitution cipher works by taking each letter in the alphabet and replacing it with another one. This was purportedly created and used by Julius Caesar. These are weak though and can be broken by:

- **Definition 2.1.4.1:** A brute-force attack tries every possible key, which for substitution ciphers becomes trivial.
- **Definition 2.1.4.2:** A frequency analysis attack matches the ciphertext to the known frequency of English letters.

Definition 2.1.4.4: Another historical example of a cipher is a Vigenere cipher. This uses a sliding password, such that repeats of the same letter are coded differently. Attacks against this have to guess possible key lengths and then try all possibilities using frequency analysis. This is much harder to break.

Definition 2.1.4.5: Another encryption method is using one time pads. Here, you select a key as long as your message. Data is converted into binary and bitwise XORed with the key. To decode it, you then bitwise XOR the ciphertext with the key. This is considered theoretically secure as long as the key generation process is truly random, the key is never reused, and the key is kept secret.

Definition 2.1.4.6: Stream ciphers use a short secret key and a nonce - a use-once but public value - to generate a long pseudorandom sequence called a keystream. Only the short secret key must be kept private. We then XOR with the message and keystream.

- To encrypt, you take the secret key K and the nonce N and feed them through a keystream generator. The resulting keystream S is then XORed with the plaintext p to produce ciphertext C .
- To decrypt, the recipient takes the same secret key K and nonce N , feeds them through the keystream generator and then XOR the result with the ciphertext C to produce the original plaintext p .

Definition 2.1.4.7: Block ciphers transform a fixed-size block of plaintext into a ciphertext block of the same size, using a secret key. Encryption applies a keyed transformation to the plaintext block, and decryption reverses the transformation using the same key. An example of this is AES encryption. To encrypt longer messages, we use a mode of operation which specifies how to encrypt message blocks in sequence - this usually involves a nonce (initialisation vector).

- **Definition 2.1.4.7:** AES Electronic Code Book (ECB) mode works by taking the plaintext, splitting it into blocks, and then encrypting each of them separately. This is not secure.
- **Definition 2.1.4.8:** AES Cipher Block Chaining (CBC) mode splits the plaintext into blocks and encrypts the first one. The result is then XORed with the second plaintext block before it is encrypted, and so on.

2.1.5 Public Key Cryptography

Definition 2.1.5.1: Rivest, Shamir, Adelman (RSA) is an algorithm where person A computes the product of two prime numbers $n = p \times q$ and makes the value of n public. Person A then generates a public value e and a private value d using p and q . e can then be used to encrypt messages to send to person A, and d used to decrypt them. This is secure based on the computational difficulty of factorisation, but with the rise of quantum computing, is becoming less secure.

Definition 2.1.5.2: Digital signatures enables others to check that a message was signed using a private key and has not been altered. To sign a message, the sender uses their private key, and then sends the message and signature to the recipient. To verify the signature, the recipient uses the sender's public key and compares the result to the message.

3 Networks

This section is taught by Dr Wanqing Tu. [Check for recommended reading.](#)

4 Databases

This section is taught by Dr Vasantam. The recommended reading for this topic is:

- Connolly, T.M. and Begg, C.E. (2015). Database Systems: A practical approach to design, implementation, and management. 6th edn. Harlow, Essex, England: Pearson Education Limited.

4.1 Enhanced Entity-Relationship Model

4.2 Semistructured Databases (XML)

4.3 XML Data Manipulation (XPath and XQuery)

4.4 Transactions

4.5 Concurrency Control

4.6 Distributed Transactions

4.7 Distributed Concurrency Control

5 Distributed Systems

This section is taught by Dr Vasantam. The recommended reading for this topic is:

- Van Steen, M. and Tanenbaum, A.S. (2025). Distributed Systems. 4th edn. [online] Self-Published. Available at: <https://www.distributed-systems.net/index.php/books/ds4/> [Accessed 5 Oct. 2026].

5.1 Introduction to Distributed Systems

5.2 Middleware and RMI Technologies

5.3 Message-Oriented Middleware and Web Services

5.4 Replication Models

5.5 Concurrent Events and Consistency Control

5.6 Fault Tolerance

5.7 Load Distribution

Part I

Appendix

6 Bibliography

6.1 Module Professors

Module Lead: Doctor Wanqing Tu

- Focus: Computer networks and wireless communications.
- Email: wanqing.tu@durham.ac.uk
- Office Hours: Thursday 11:00 - 12:00.

Lecturer: Doctor Thirupathaiah Vasantam

- Focus: Applied probability and performance modelling.
- Email: thirupathaiah.vasantam@durham.ac.uk
- Office Hours: Monday 14:00 - 15:00.

Lecturer: Doctor Carlton Shepherd

- Focus: Cyber security.
- Email: carlton.g.shepherd@durham.ac.uk
- Office Hours: Email to make an appointment.

6.2 References

6.3 List of Figures

6.4 Reading List

6.4.1 Cyber Security

Nakov, S., Stefanov, M. and Shideroff, M. (2018). Practical Cryptography for Developers. [online] sec. Encryption: Symmetric and Asymmetric Available at: <https://cryptobook.nakov.com/> [Accessed 9 Oct. 2026].

6.4.2 Databases

Connolly, T.M. and Begg, C.E. (2015). Database Systems: A practical approach to design, implementation, and management. 6th edn. Harlow, Essex, England: Pearson Education Limited.

6.4.3 Distributed Systems

Van Steen, M. and Tanenbaum, A.S. (2025). Distributed Systems. 4th edn. [online] Self-Published. Available at: <https://www.distributed-systems.net/index.php/books/ds4/> [Accessed 5 Oct. 2026].